



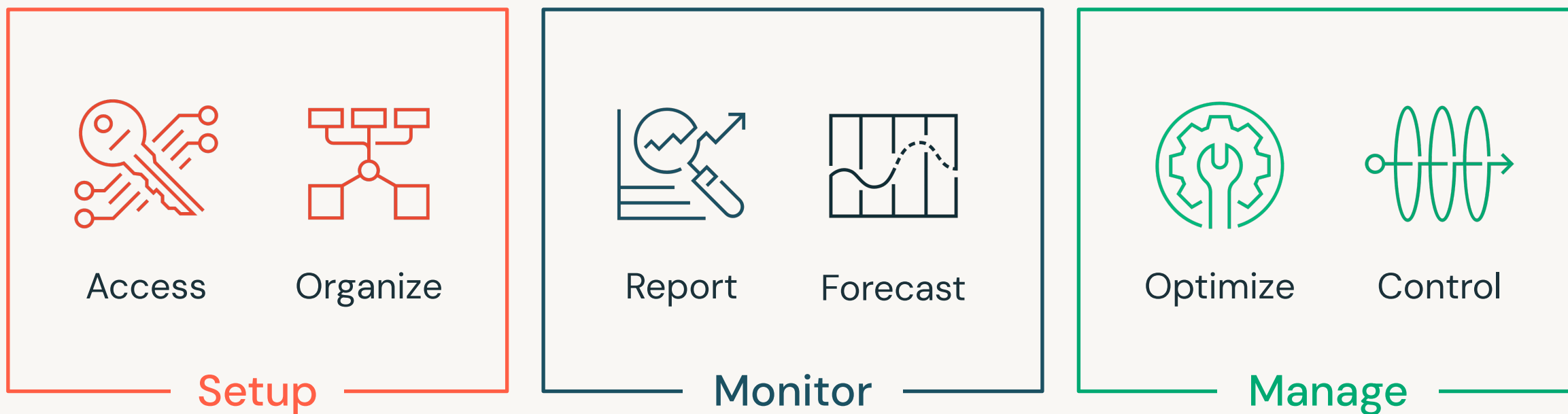
FinOps

Cost management & value tracking

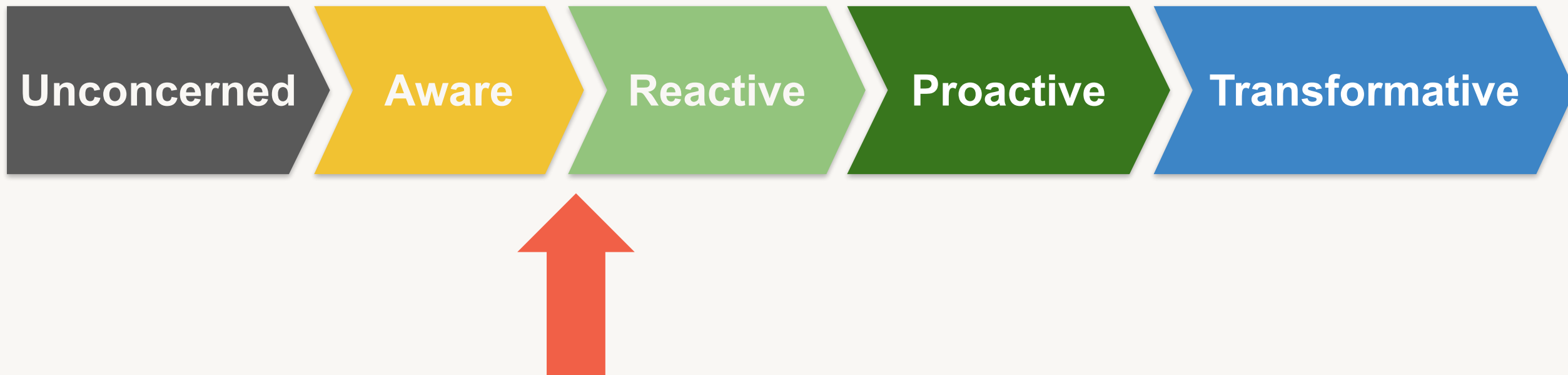
May 2026



Steps to cost management success



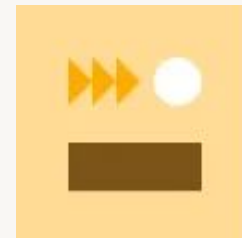
Journey of Managing Costs



Databricks Pricing Model

Pricing model

Let's start with what you don't pay for



Databricks uses a flexible **pay-as-you-go** pricing model that scales elastically with use.

No costs for:

- License, startup or monthly fee
- Workspace usage with no compute running
- Data catalog (Unity Catalog)

General concepts

Workspaces and compute instances



Databricks workspaces have different tiers affecting the price:

- Standard
- Premium
- Enterprise (AWS only)

Databricks charges for usage in **Databricks Units (DBUs)**. The number of DBUs a workload consumes depend on:

- Cluster/warehouse size
- Photon acceleration
- Time the compute instance is running

4 Workers

122 GB Memory
16 Cores

1 Driver

30.5 GB Memory, 4 Cores

Runtime

13.1.x-scala2.12

Unity Catalog

Photon

i3.xlarge

10 DBU/h



DBU pricing

DBU prices depend on several factors:

- SKU (type of compute)
- Cloud provider
- Cloud region
- Workspace tier

Workflows & Streaming

Jobs

Starting at **\$0.07 / DBU**

Run data engineering pipelines to build data lakes and manage data at scale

[Learn more →](#)

Workflows & Streaming

Delta Live Tables

Starting at **\$0.20 / DBU**

Easily build high-quality streaming or batch ETL pipelines using Python or SQL with the DLT edition that is best for your workload

[Learn more →](#)

Data Warehousing

Databricks SQL

Starting at **\$0.22 / DBU**

Run SQL queries for BI reporting, analytics and visualization to get timely insights from data lakes. Available in both Classic and Serverless (managed) Compute.

[Learn more →](#)

Data Science & Machine Learning

All Purpose Compute for Interactive Workloads

Starting at **\$0.40 / DBU**

Run interactive data science and machine learning workloads. Also good for data engineering, BI and data analytics

[Learn more →](#)

Data Science & Machine Learning

Serverless Real-Time Inference

Starting at **\$0.07 / DBU**

Make live predictions in your apps and websites.

[Learn more →](#)

Databricks Platform & Add-Ons

Databricks Platform & Add-Ons

Cross-platform capabilities that provide the right level of management, governance and security to run everything from basic to enterprise-critical workloads

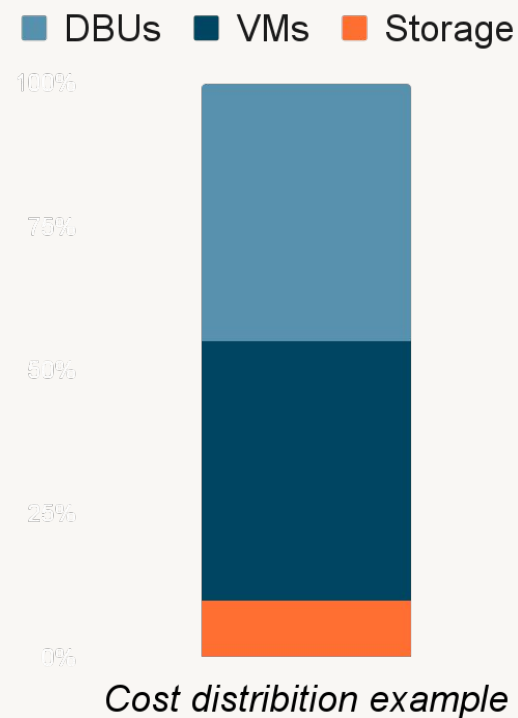
[Learn more →](#)

Total cost

DBUs + cloud infrastructure



- DBUs
- Cloud compute
 - VMs
 - VM disks
 - Networking
- Cloud storage (ADLS, S3, GCS)
 - Managed Databricks storage (DBFS)
- Custom cloud services (optional)
 - Private Endpoints
 - Firewalls



Serverless compute

VM costs integrated in DBUs



Managing Your Costs



Report

Explore your data, gain insights and share them with relevant stakeholders

Billing and cost analysis

Cloud provider overview

AWS



Billing

DBUs: Via AWS Marketplace (total DBU costs) or directly via DB

Infra: AWS bill

Cost analysis

DBUs: Account Console, Billable Usage Logs

Infra: AWS Cost Explorer (tags)

Azure



Billing

Same bill as for other Azure services – first party service

Cost analysis

Azure Cost Management tool

GCP



Billing

DBUs: Via GCP bill (total DBU costs) or directly via DB

Infra: GCP bill

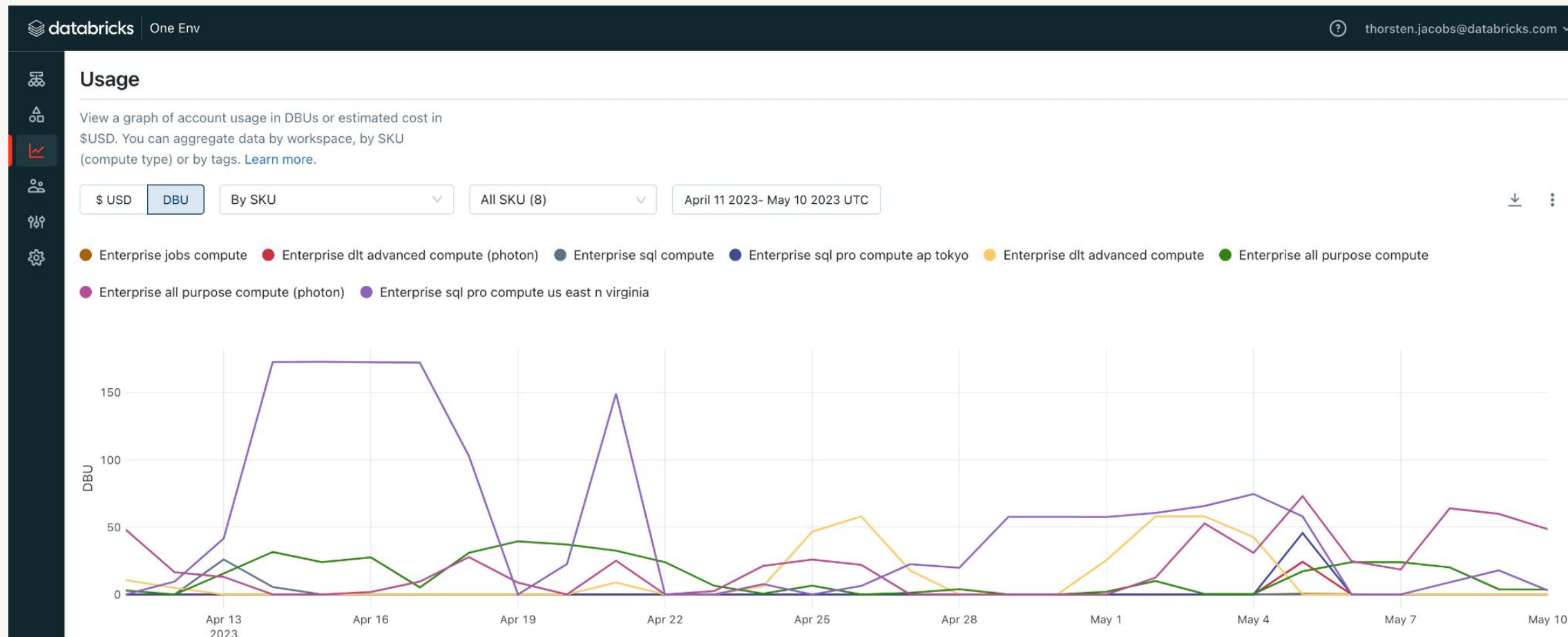
Cost analysis

DBUs: DB Account Console, Billable Usage Logs

Infra: Big Query (tags)

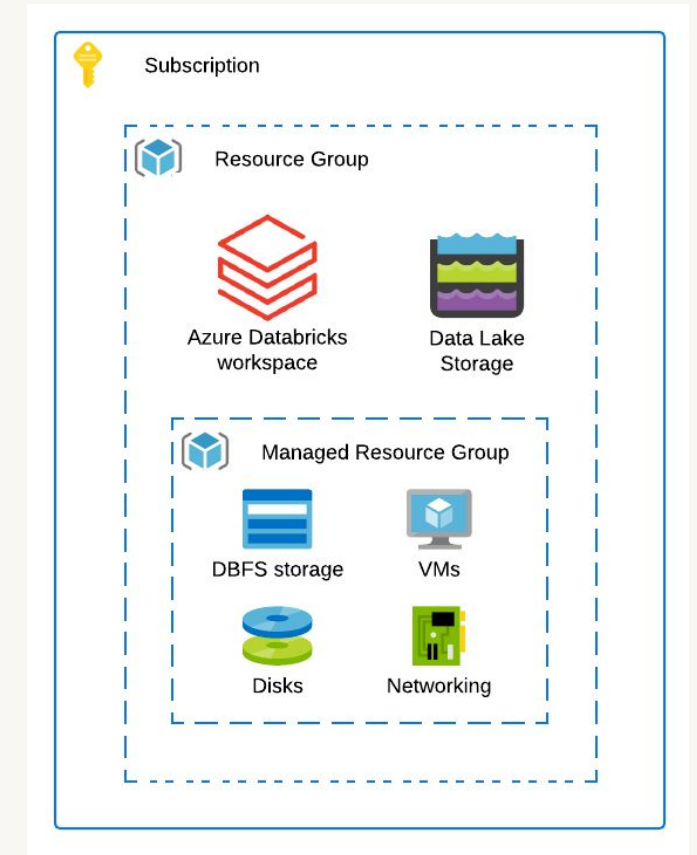
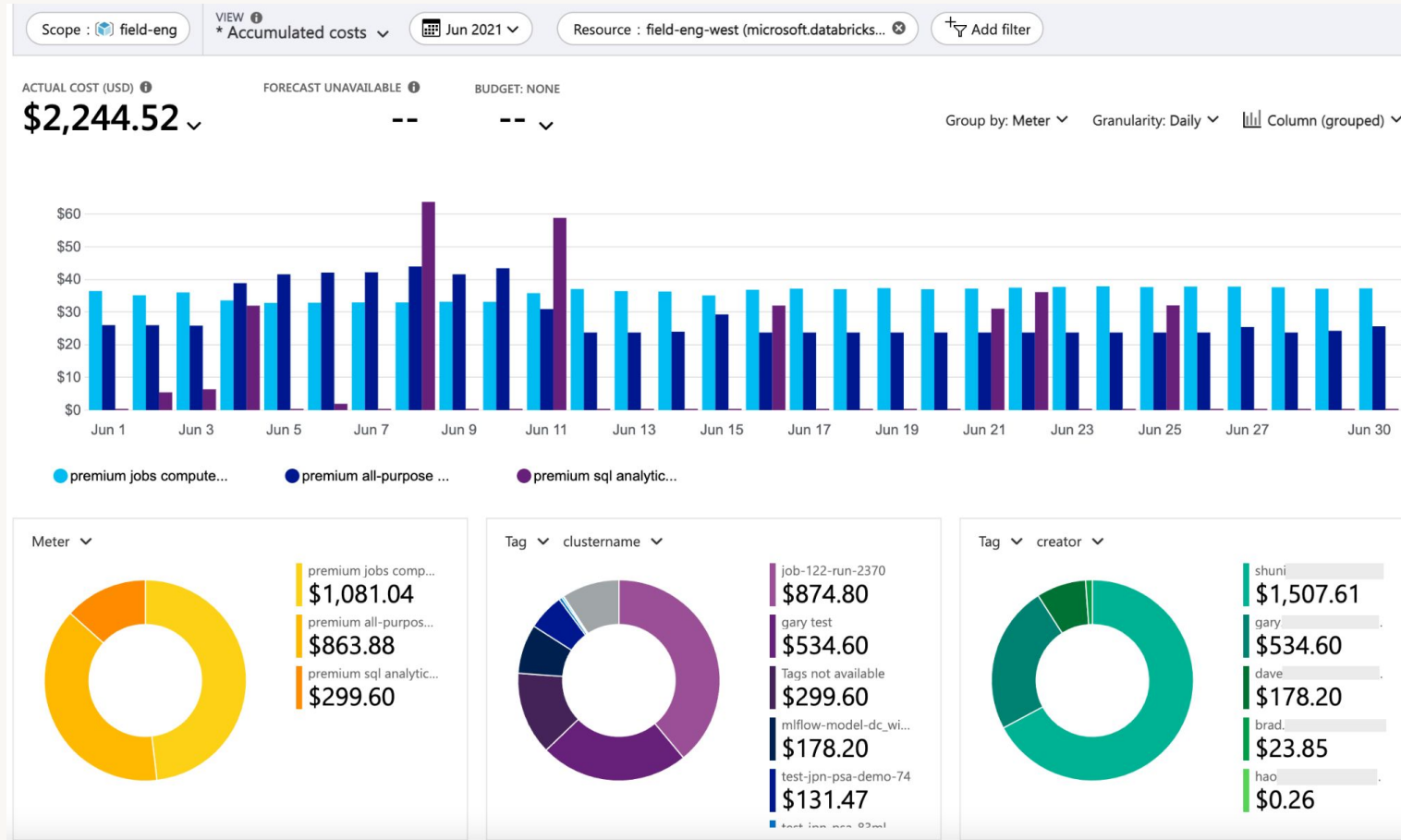
Databricks Usage: Account Console

Visualizes Billable Usage Logs, see DBU usage per workspace or SKU



Example: Azure Cost Management Tool

See DBU and infrastructure costs, break down by SKU or tag



How to Best Access Cost Data

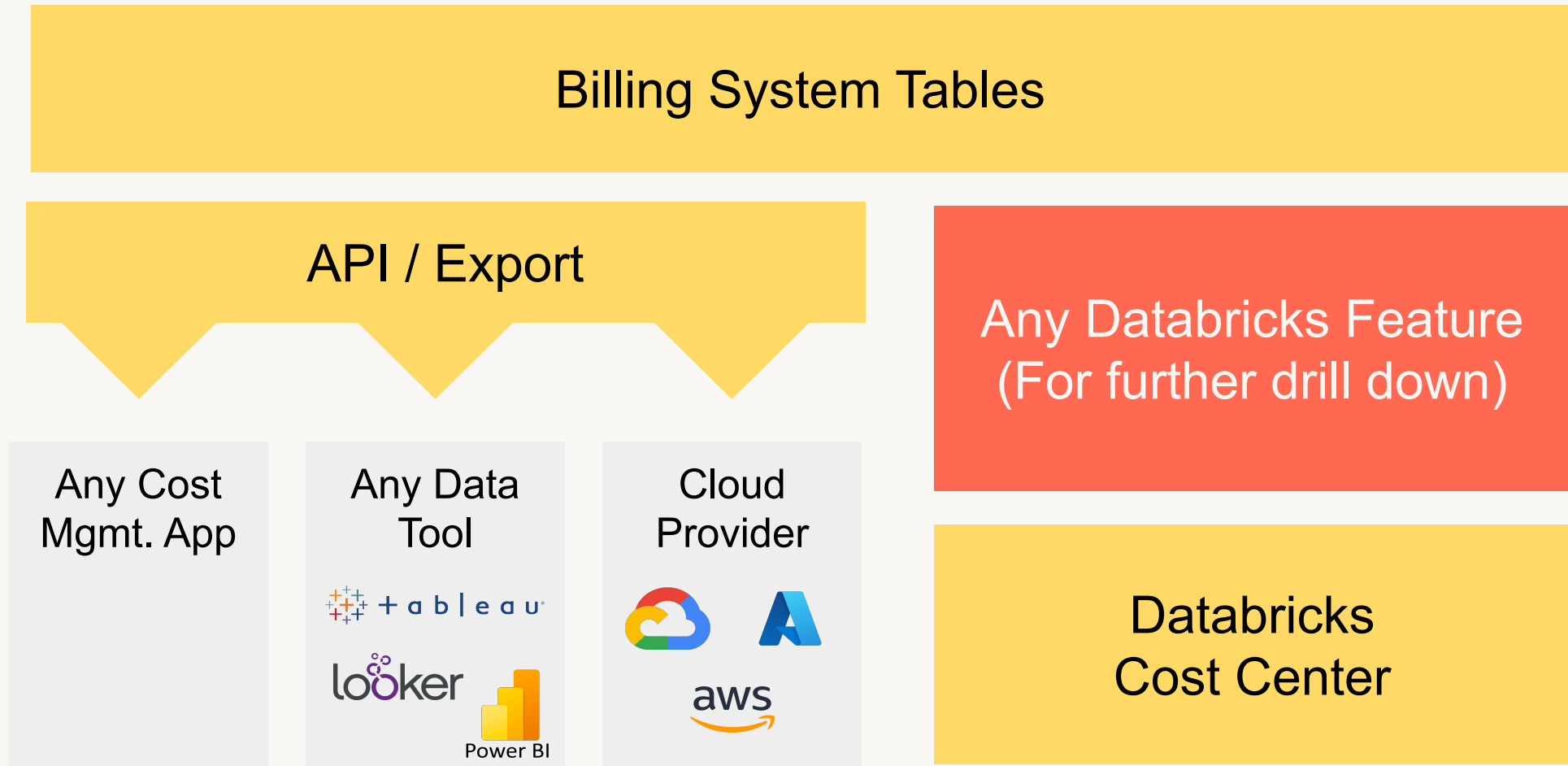
Existing Methods

- Cloud Provider's Cost Tools
- Billable Usage Logs CSV
- Account Console UI
- Overwatch – Databricks Labs

New Recommended Method

- Billing System Tables

Vision for The Databricks Cost Management Toolset



Billable usage

Analyze and optimize DBU consumption by leveraging usage tables

- What is the daily trend in DBU consumption?
- How many DBUs of each SKU have been used throughout this month?
- How much of each SKU did a workspace use on June 1?
- Which jobs consumed the most DBUs?
- How much usage can be attributed to resources with a certain tag?
- Show me the SKUs where usage is growing
- What is the usage trend of All Purpose Compute (Photon)?

Catalogs > system > billing >

system.billing.usage

Tags: Add

Owner: System user Popularity: Size: Unknown

Columns Sample Data Details Permissions History Lineage Insights Quality

Column	Type
account_id	string
workspace_id	string
usage_record_id	string
sku_name	string
cloud	string
usage_start_time	timestamp
usage_end_time	timestamp
usage_date	date
custom_tags	map
usage_unit	string
usage_quantity	decimal
system_metadata	struct

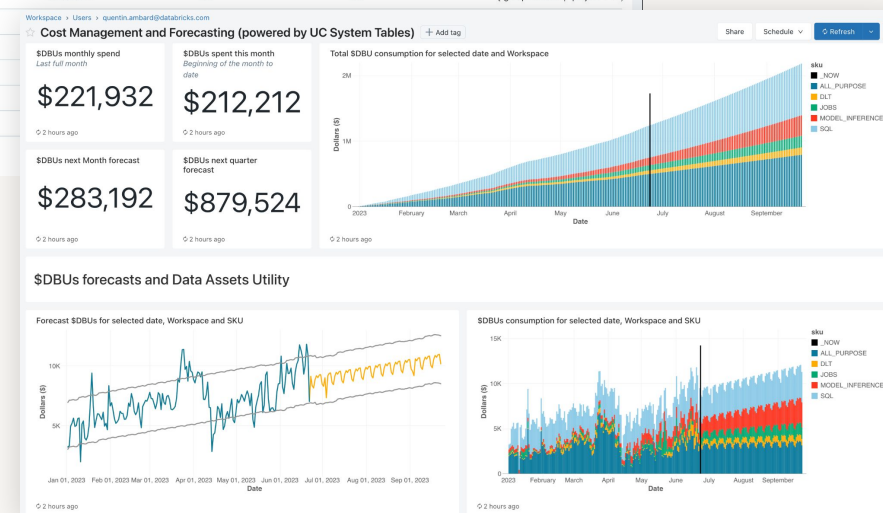
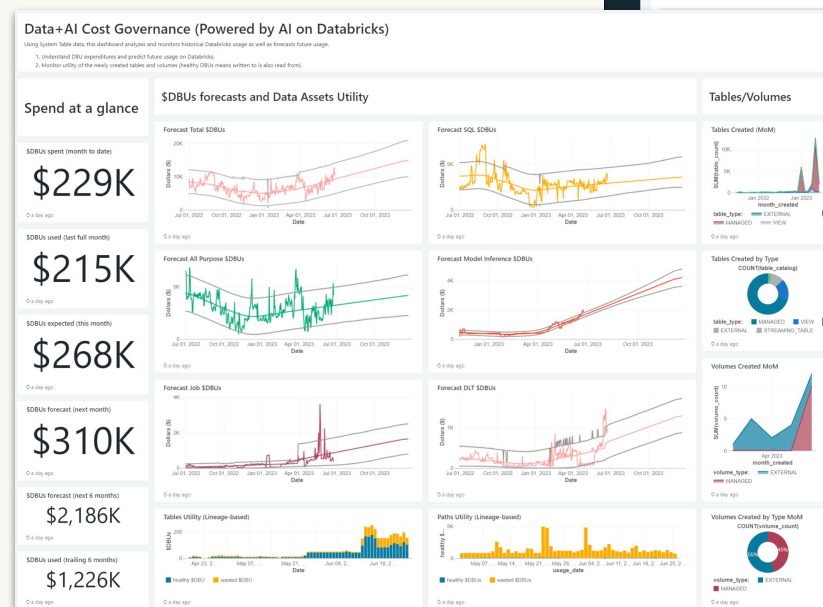
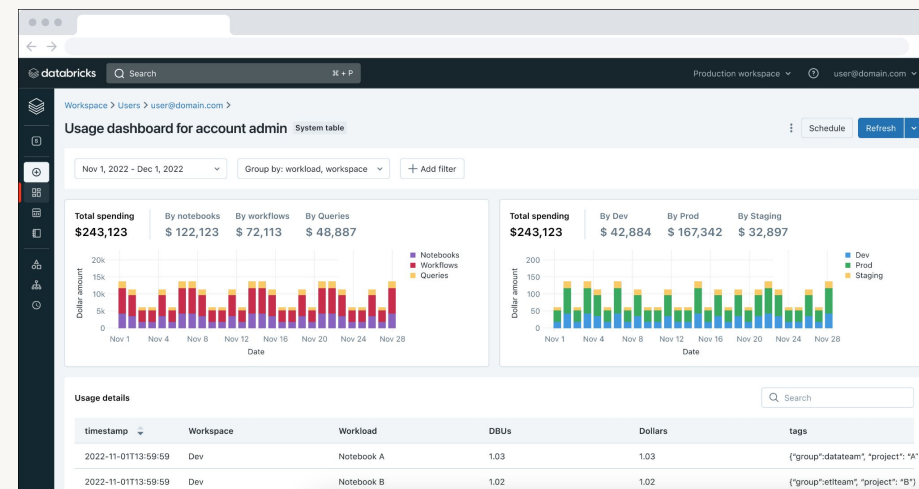


Column name	Data type	Description	Example
record_id	string	Unique ID for this record	11e22ba4-87b9-4cc2-9770-d10b894b7118
account_id	string	ID of the account this report was generated for	23e22ba4-87b9-4cc2-9770-d10b894b7118
workspace_id	string	ID of the Workspace this usage was associated with	1234567890123456
sku_name	string	Name of the SKU	STANDARD_ALL_PURPOSE_COMPUTE
cloud	string	Cloud this usage is relevant for. Possible values are <code>AWS</code> , <code>AZURE</code> , and <code>GCP</code> .	AWS AZURE GCP
usage_start_time	timestamp	The start time relevant to this usage record	2023-01-09 10:00:00.000
usage_end_time	timestamp	The end time relevant to this usage record	2023-01-09 11:00:00.000
usage_date	date	Date of the usage record, this field can be used for faster aggregation by date	2023-01-01
custom_tags	map	Tags applied by the users to this usage	{ "env": "production" }
usage_unit	string	Unit this usage is measured in. Possible values include DBUs.	DBU
usage_quantity	decimal	Number of units consumed for this record.	259.2958
usage_metadata	struct	System-provided metadata about the usage, such as related IDs or configuration parameters	{cluster_id: 12345; instance_pool_id: null; warehouse_id: null; job_id: null; node_type: null}

System Tables in Action

Setup a dashboard using Databricks to keep an eye on costs

Slice the data however you like using SQL, Python or other supported languages.



Audit Events

Audit Log events in System tables helps you to answer common account usage questions

- Who accessed this table?
- Which users accessed a table within the last day?
- Which tables did a user access?
- View all permissions changes
- View the most recently run notebook commands

Catalogs > system > access >

system.access.audit

Tags:

Owner: System user  Popularity:  Size: Unknown

Columns Sample Data Details Permissions History Lineage Insights Quality

Column	Type
account_id	string
workspace_id	string
version	string
event_time	timestamp
event_date	date
source_ip_address	string
user_agent	string
session_id	string
user_identity	struct
service_name	string
action_name	string



Table and Column Lineage

Build on Unity Catalog's data lineage feature, allowing you to programmatically query lineage data to fuel decision making and reports.

There are two lineage system tables:

- `system.access.table_lineage`
- `system.access.column_lineage`

The screenshot displays two side-by-side views of the Databricks Unity Catalog interface. Both views show the 'system.access' catalog and the 'access' schema. The left view is for the 'system.access.table_lineage' table, and the right view is for the 'system.access.column_lineage' table. Both tables have a 'Columns' tab selected, showing a list of columns and their types. The 'system.access.table_lineage' table has 13 columns, and the 'system.access.column_lineage' table has 13 columns. The 'system.access.column_lineage' table includes an additional column, 'source_column_name', which is of type 'string'.

Column	Type
account_id	string
metastore_id	string
workspace_id	string
entity_type	string
entity_id	string
entity_run_id	string
source_table_full_name	string
source_table_catalog	string
source_table_schema	string
source_table_name	string
source_path	string
source_type	string
target_table_full_name	

Column	Type
account_id	string
metastore_id	string
workspace_id	string
entity_type	string
entity_id	string
entity_run_id	string
source_table_full_name	string
source_table_catalog	string
source_table_schema	string
source_table_name	string
source_path	string
source_type	string
source_column_name	string

Clusters

Understand infrastructure usage and scaling events

- How long were clusters up ?
- What type of VM types were used ?
- How did they scale up / down

Schemas

- `system.compute.clusters`
- `system.compute.node_timeline`
- `system.compute.node_types`
- `system.compute.warehouse_events`

The screenshot shows the Databricks Catalog Explorer interface. The left sidebar displays a tree view of the catalog structure, with 'system' expanded to show 'compute' and its sub-schemas. The main panel shows the 'system.compute.clusters' schema details, including the owner 'System user', popularity, and a list of columns with their types and comments.

Column	Type	Comment
account_id	string	
workspace_id	string	
cluster_id	string	
cluster_name	string	
owned_by	string	
create_time	timestamp	
delete_time	timestamp	
driver_node_type	string	
worker_node_type	string	
worker_count	long	
min_autoscale_workers	long	
max_autoscale_workers	long	



Query

Understand how data is being queried and by whom

- What tables/column are most frequently used ?
- How are tables being queried ?
- Who looks at what data ?

The screenshot shows the Databricks Catalog Explorer interface. On the left, a sidebar displays a tree view of the catalog structure under 'sys'. The main panel on the right shows the 'system.query.history' table. The table has columns for 'Column', 'Type', and 'Comment'. The table is filtered to show only 'string' types. The table is titled 'system.query.history' and is owned by 'System user'. The table has a popularity score of 1. The table is filtered to show only 'string' types. The table is titled 'system.query.history' and is owned by 'System user'. The table has a popularity score of 1. The table is filtered to show only 'string' types.

Column	Type	Comment
account_id	string	
workspace_id	string	
statement_id	string	
executed_by	string	
session_id	string	
execution_status	string	
warehouse_id	string	
executed_by_user_id	string	
statement_text	string	
statement_type	string	
error_message	string	
warehouse_channel	string	



System Tables – Billing Forecast demo

Observability with Unity Catalog

System Tables: Billing Forecast, Usage Analytics, and Access Auditing With Databricks Unity Catalog

DEMO TYPE

Product Tutorial

DURATION

Self-paced

RELATED CONTENT

[View all tutorials →](#)

[Databricks Unity Catalog: File](#)

What You'll Learn

Databricks Unity Catalog is the world's first AI-powered governance solution for the lakehouse. It empowers enterprises to seamlessly govern their structured and unstructured data, ML models, notebooks, dashboards, and files on any cloud or platform.

Through Delta Sharing, Databricks Unity Catalog offers direct access to many of the lakehouse activity logs exposed in Delta as System Tables. System Tables are the cornerstone of lakehouse observability and enable at-scale operational intelligence on numerous key business questions. In this demo, we'll show how Unity Catalog System Tables can be used to:

To install the demo, get a **free Databricks workspace** and execute the following two commands in a Python notebook

```
%pip install dbdemos
```

```
import dbdemos  
dbdemos.install('uc-04-system-tables')
```

Recommended



TUTORIAL

[Table ACL and Dynamic Views With UC](#)

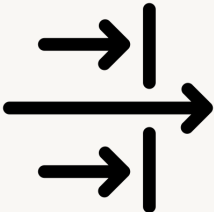


TUTORIAL

[Access Data on External Locations](#)

https://www.databricks.com/resources/demos/tutorials/governance/system-tables?itm_data=demo_center





Control

Setup governance that gives you confidence

Control Spend

1. Cluster Policies
2. Share usage info with users
3. Budgets and Alerts

Demo: Budget Alerting on System Tables

The screenshot displays the Databricks SQL Editor interface. The top navigation bar includes the 'Billing DBUs' tab, a search bar, and user information 'alex.lichen@ databricks.com'. The left sidebar shows a 'Schema browser' with a tree view of tables and views under 'hive_metastore > default'. The main area contains a query editor with the following SQL code:

```
1 SELECT *
2 FROM system.billing.billing_logs
```

Below the query editor, the 'Results' section shows a table with 8 rows and 6 columns: #, account_id, workspace_id, timestamp, date, and cluster_id. The first row of data is:

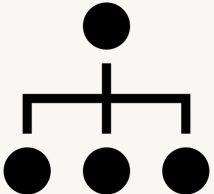
#	account_id	workspace_id	timestamp	date	cluster_id
1	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-25 14:59:59.000	2022-06-25	0625-141012-wz7ki0xb
2	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-28 02:59:59.000	2022-06-28	0628-021018-y9ofklhr
3	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-28 04:59:59.000	2022-06-28	0628-045018-fvxizzg1
4	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-29 02:59:59.000	2022-06-29	0629-025012-exdiybnz
5	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-27 03:59:59.000	2022-06-27	0627-035022-3yrehhn4
6	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-30 09:59:59.000	2022-06-30	0630-091010-6vnl9tcd
7	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-26 09:59:59.000	2022-06-26	0626-091011-qu1aknq6
8	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-30 01:59:59.000	2022-06-30	0630-012013-qepqveuw

At the bottom of the results section, it indicates '4.19 s runtime | 1K rows' and 'Refreshed 2 minutes ago'.

Easily set up alerts on usage data

Supported Destinations:

- Slack
- Teams
- PagerDuty
- Email
- Webhook

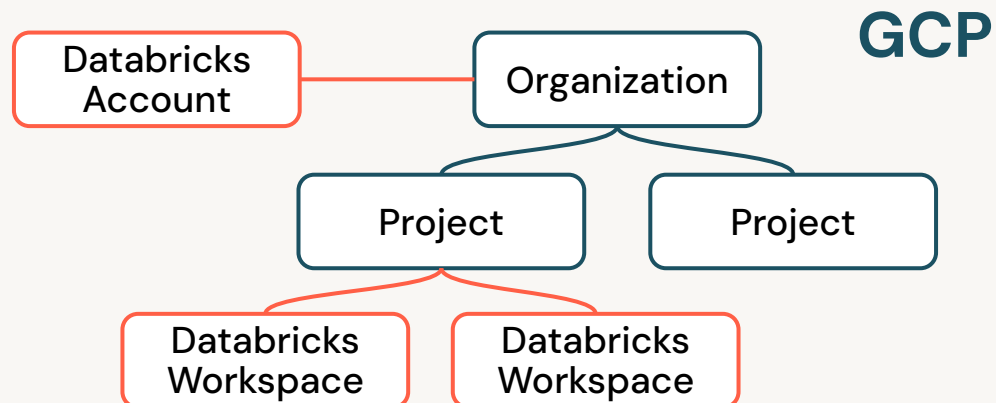
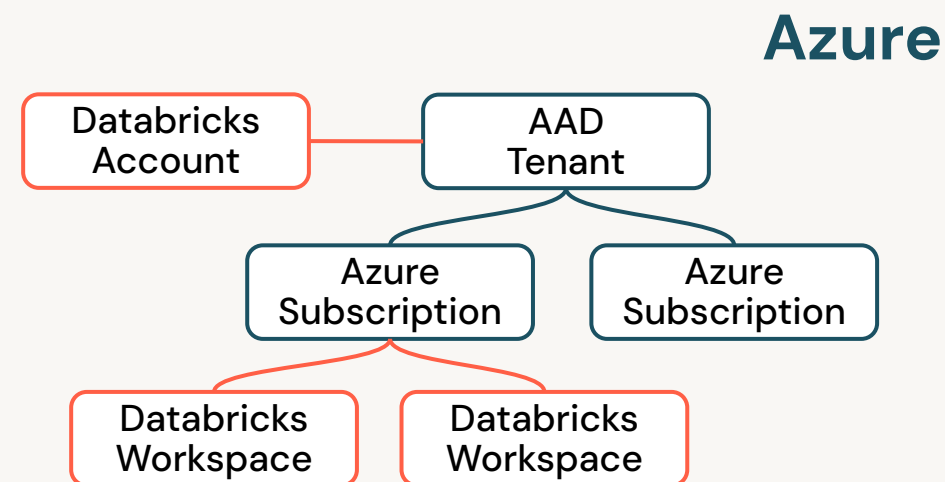
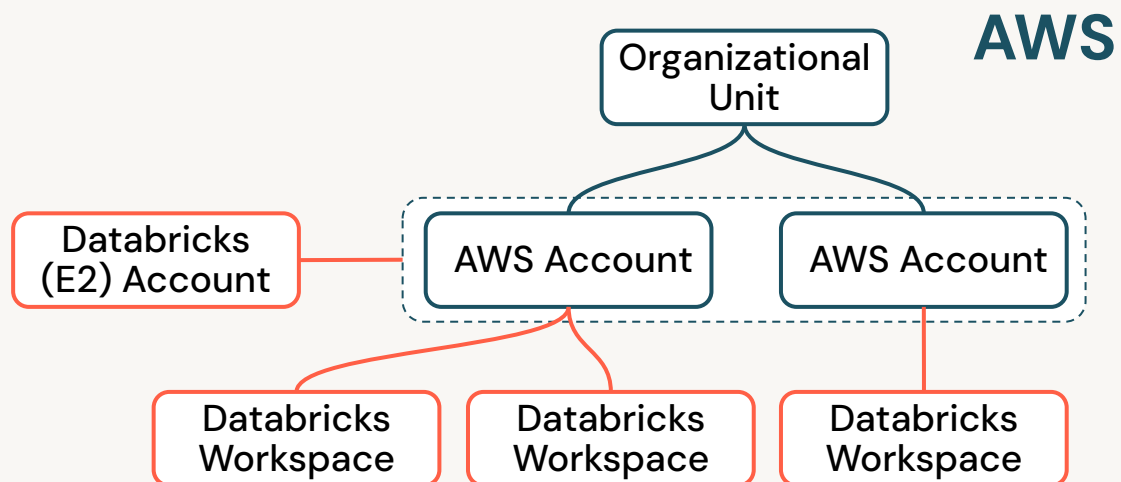


Organize

Organize costs according to your needs. By team, cost center, environment, application, etc.

Cloud Provider Hierarchy

Consider hierarchy for cost separation



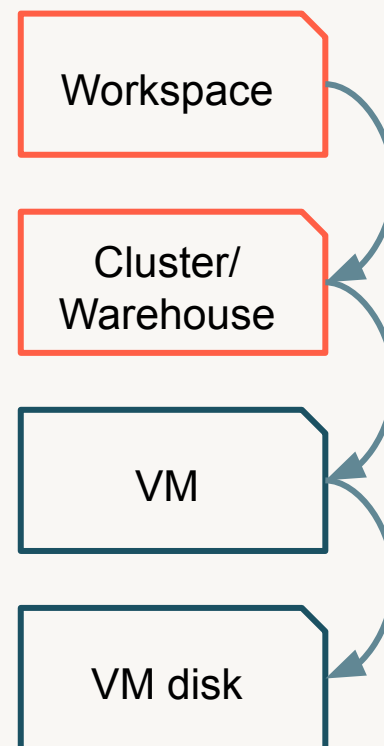
Resource tagging

Fine grained cost monitoring

Tags: Key/value pairs,
default + custom tags

Databricks tags

- Workspace
- Cluster or SQL warehouse
- Pools



Tags are inherited by
associated resources.

Propagate to DBU
usage logs and cloud
provider tools.

Chargeback / Showback

1. Create separate workspaces for each team or project or use cluster tags
2. Break down cost per workspace or tag with methods described earlier
3. Create automated process to deliver report to the respective unit

The same approach can be used for ROI calculations of workloads or projects.

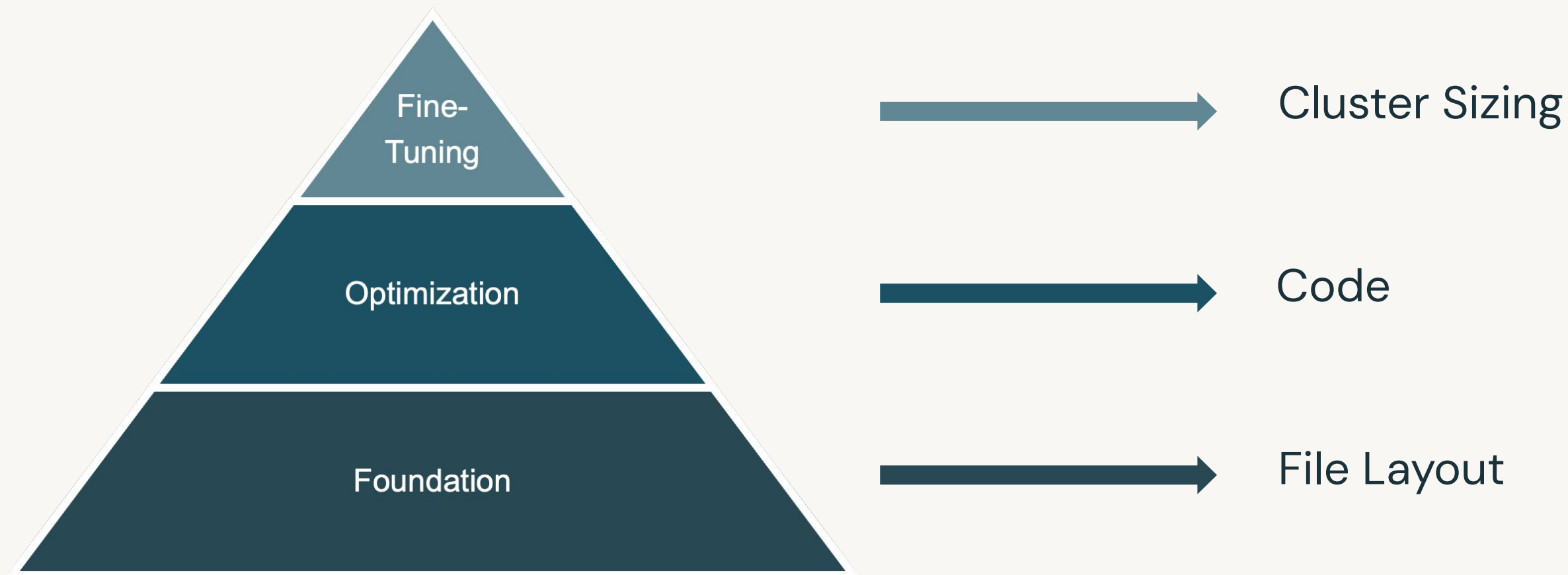


Optimize

Fine tune your environment to get more value for your investment

Performance optimizations

Optimized platform → Less compute needed
+ shorter processing time → Reduced costs



Optimize – File layout and Storage



- Use Delta format and **Managed Tables** for best performance
- Optimize file layout by following Delta best practises
 - Compact files (OPTIMIZE)
 - Read less data with data skipping (liquid clustering/z-ordering)
 - Use Unity Catalog managed tables for automatic file size tuning
- Avoid re-processing large tables by using a CDC architecture (DLT)
- Run VACUUM on Delta tables to remove redundant version history

Optimize – Code



- Use latest runtime version for best performance
- Know your Spark, make sure your code parallelizes well
 - Avoid non-vectorized user defined functions (UDFs) and loops
→ Use built-in functions and functional programming whenever possible
 - Be aware of Spark's lazy evaluation
 - Clean out old Spark configurations
- Use **SQL (declarative language)** → Express what you want to do and let the Spark engine take care of how

Optimize – Compute



- Make it simple: go **Serverless!**
- Select the optimal compute for your workload
 - Automated workflows on job clusters
 - SQL warehouses for SQL workloads
 - Single node clusters for workloads that do not parallelize (e.g. pure Python)
- Minimize idle time
 - Auto termination and auto scaling
 - Serverless: Instantly available + faster autoscaling
- Consider spot instances to save VM costs

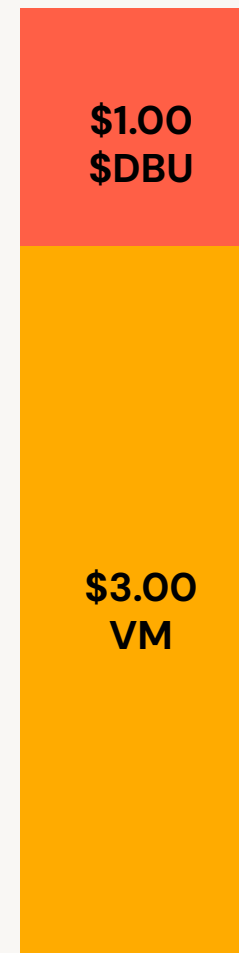
Optimize – Compute

Give your jobs horsepower

Larger cluster, Photon runtime, Disk Cache

- Larger or more powerful compute resources do not necessarily mean higher costs
- You only pay for the time the cluster is used:
Faster execution → Shorter billing period
- 2x faster means that you spend half on the infrastructure costs

TCO \$4.00



Spark

26% Savings



TCO \$2.95



Photon

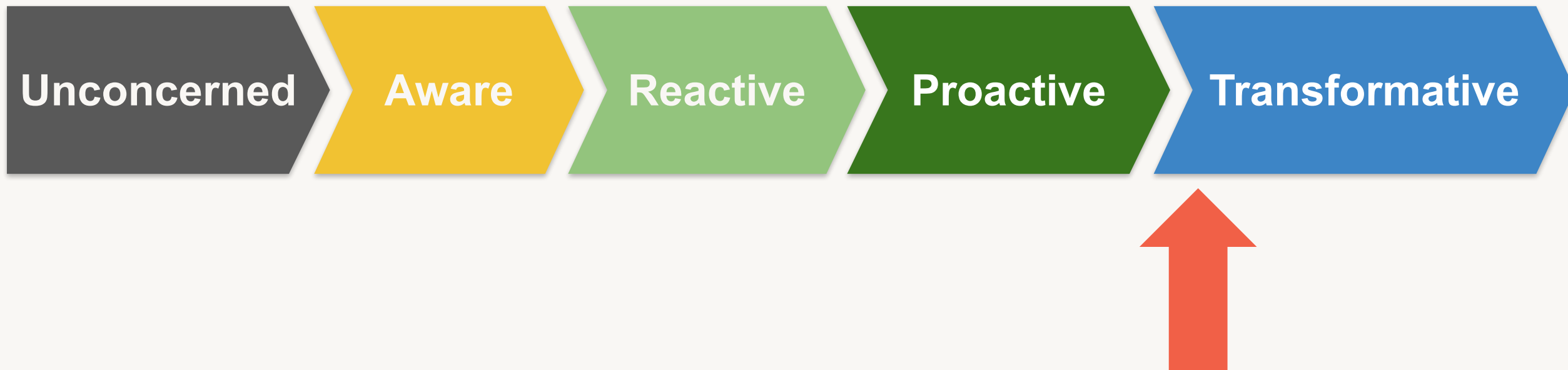
Summary – Cost optimization



Key points to stay under budget

- Foundation: Avoid vendor lock-in by choosing an architecture based on open standards and simplify your architecture with the Lakehouse
- Optimize your platform for performance
- Use reporting to understand your costs
- Set up methods to control your spend (cluster policies, alerts)
- Optional: Forecast spend and get discounts via DBU commit

Journey of Managing Costs



Resources

- [Databricks pricing](#)
- [Best Practices for Cost Management on Databricks](#)

Key strategies for cost management on their platform

1. Utilizing **Cluster Policies** to control who can create clusters and define the size and scope of those clusters.
 2. Designing environments to minimize non-DBU costs generated by Databricks workspaces, such as **storage and networking costs**.
 3. **Monitoring** costs with Databricks' options for automated notifications and alerts based on usage analytics.
- [Comprehensive Guide to Optimize Databricks, Spark and Delta Lake Workloads](#)



The following section is screenshots of the demos to have as a backup if we have wifi issues with the recording or live demo

Reporting

Cost Management Demo on Usage Table

Python ▾

File Edit View Run Help Last edit was 1 hour ago Provide feedback

▶ Run all

● greg.kroleski@ databricks... ▾

📅 Schedule

Share



Workspace



← greg.kroleski@databricks....



Sort Name ▾



📄 Cost Management Demo on Usa...

📄 Enable System Tables

Cmd 1

What is the daily trend in DBU consumption?

SQL



```
1 %sql
2
3 SELECT usage_date, sum(usage_quantity) as DBUs
4   FROM system.billing.usage
5  WHERE year(usage_date) = year(CURRENT_DATE)
6  GROUP BY usage_date
7  ORDER BY usage_date ASC
```

▶ (2) Spark Jobs

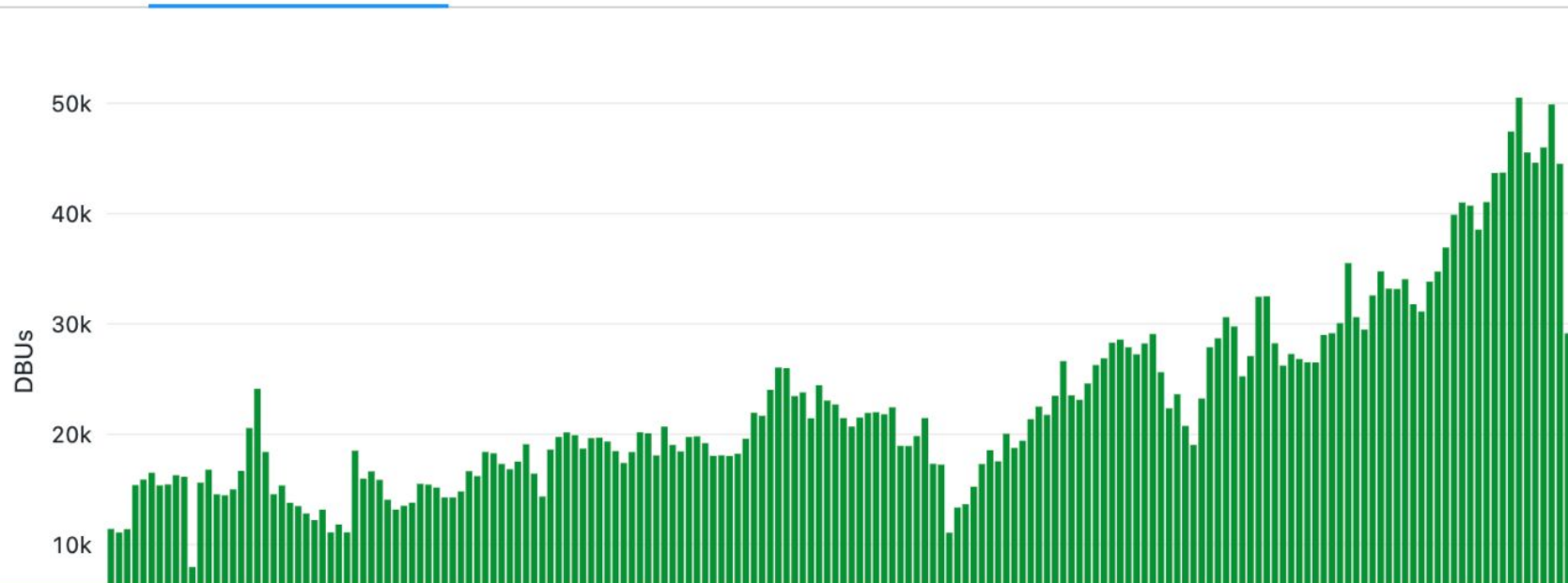
▶ 📄 _sqlidf: pyspark.sql.dataframe.DataFrame = [usage_date: date, DBUs: decimal(38,18)]

Table

Visualization 1 ▾



Preview



Provide feedback [🔗](#)

How many DBUs of each SKU have been used so far this month?

```
1 %sql
2
3 SELECT sku_name, usage_date, sum(usage_quantity) as DBUs
4 | FROM system.billing.usage
5 WHERE
6 | month(usage_date) = month(CURRENT_DATE)
7 GROUP BY sku_name, usage_date
```

► (2) Spark Jobs

►  _sqldf: pyspark.sql.dataframe.DataFrame = [sku_name: string, usage_date: date ... 1 more field]

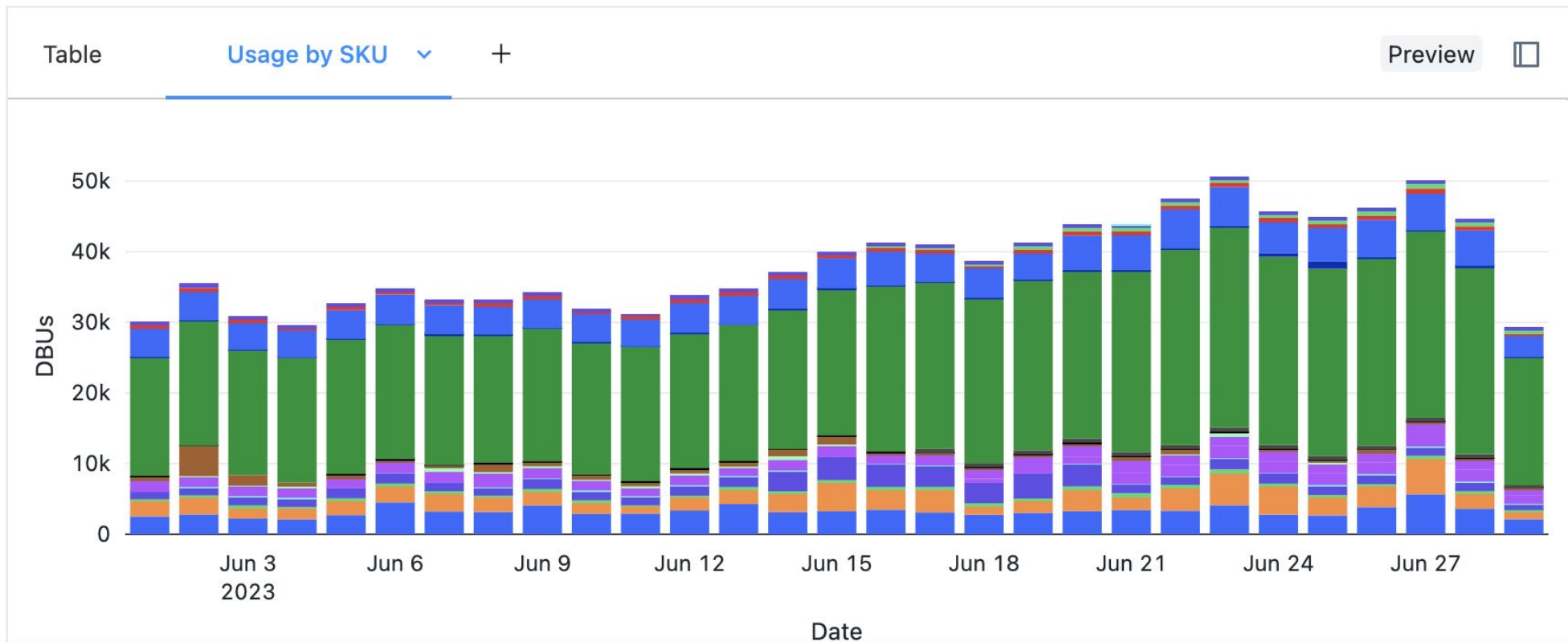
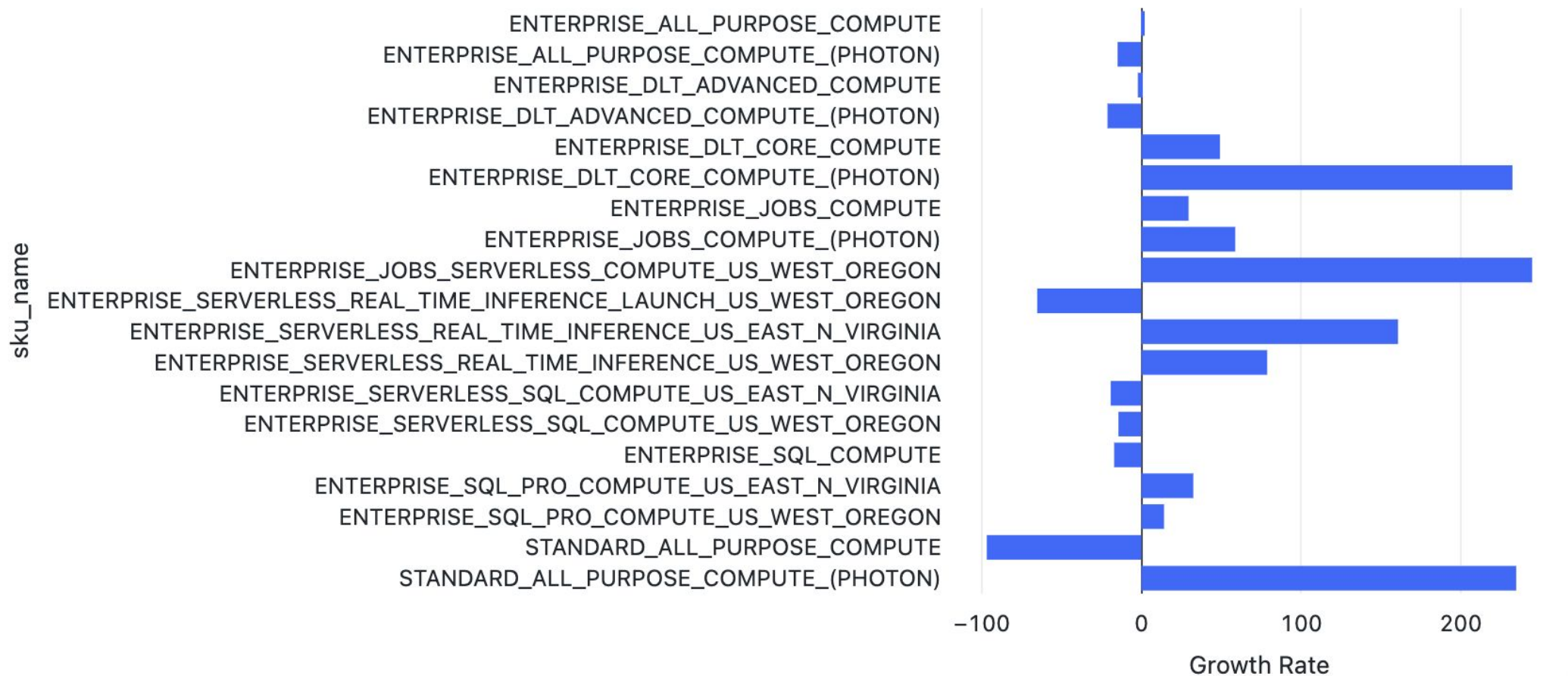


Table ▾

Visualization 1 ▾

+

Preview



Edit Visualization

19 rows

Refreshed 1 hour ago

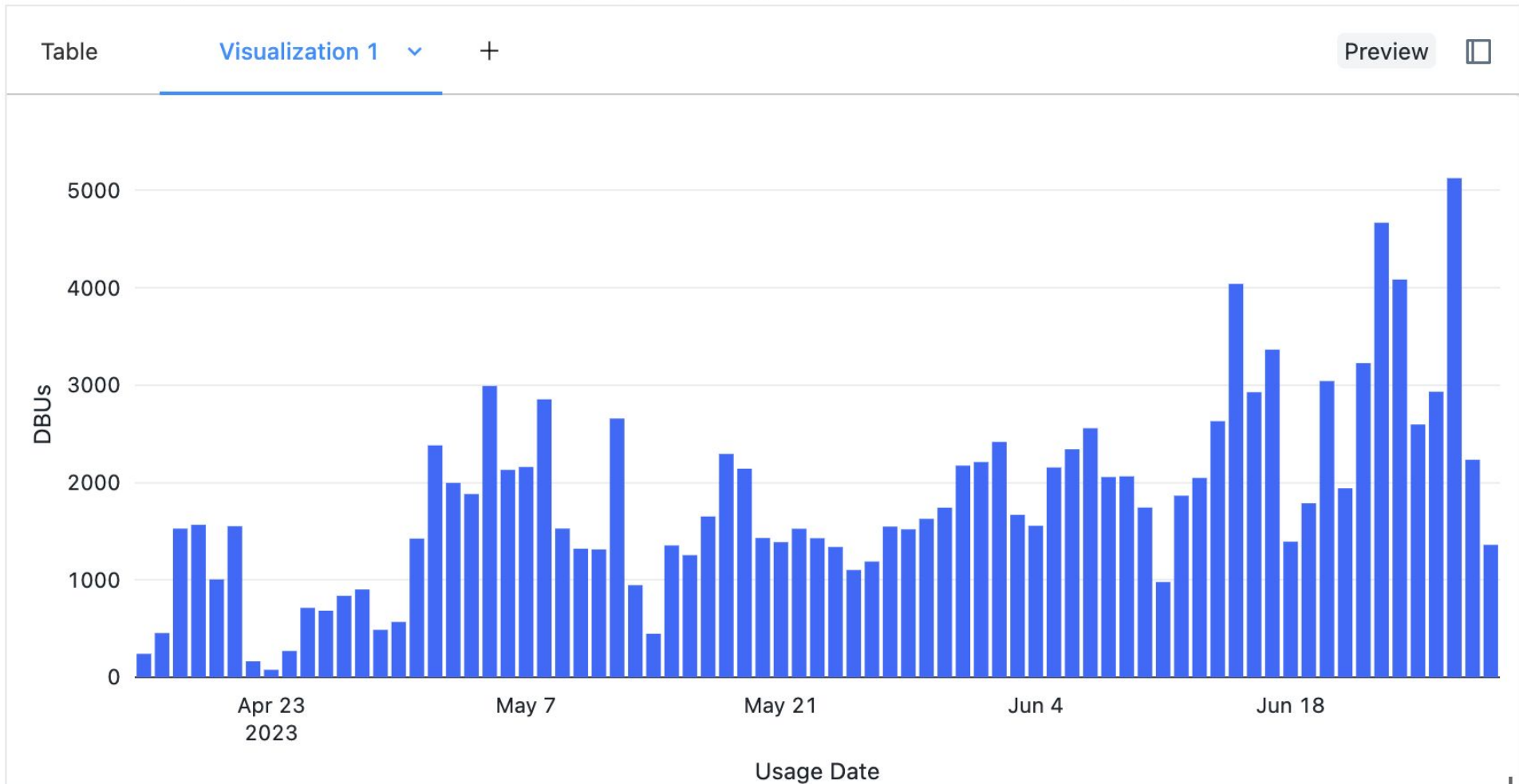
i This result is stored as `_sqlddf` and in the IPython output cache as `Out[48]`. [Learn more](#)

Command took 3.03 seconds -- by greg.kroleski@databricks.com at 6/29/2023, 1:59:16 PM on greg.kroleski@databricks.com's Personal Compute Cluster

```
2
3  SELECT sku_name, usage_date, sum(usage_quantity) as DBUs
4      FROM system.billing.usage
5  WHERE year(usage_date) = year(CURRENT_DATE)
6  AND sku_name = "ENTERPRISE_ALL_PURPOSE_COMPUTE_(PHOTON)"
7  AND usage_date > "2023-04-15"
8  GROUP BY sku_name, usage_date
```

► (2) Spark Jobs

►  _sqldf: pyspark.sql.dataframe.DataFrame = [sku_name: string, usage_date: date ... 1 more field]



▸ (2) Spark Jobs

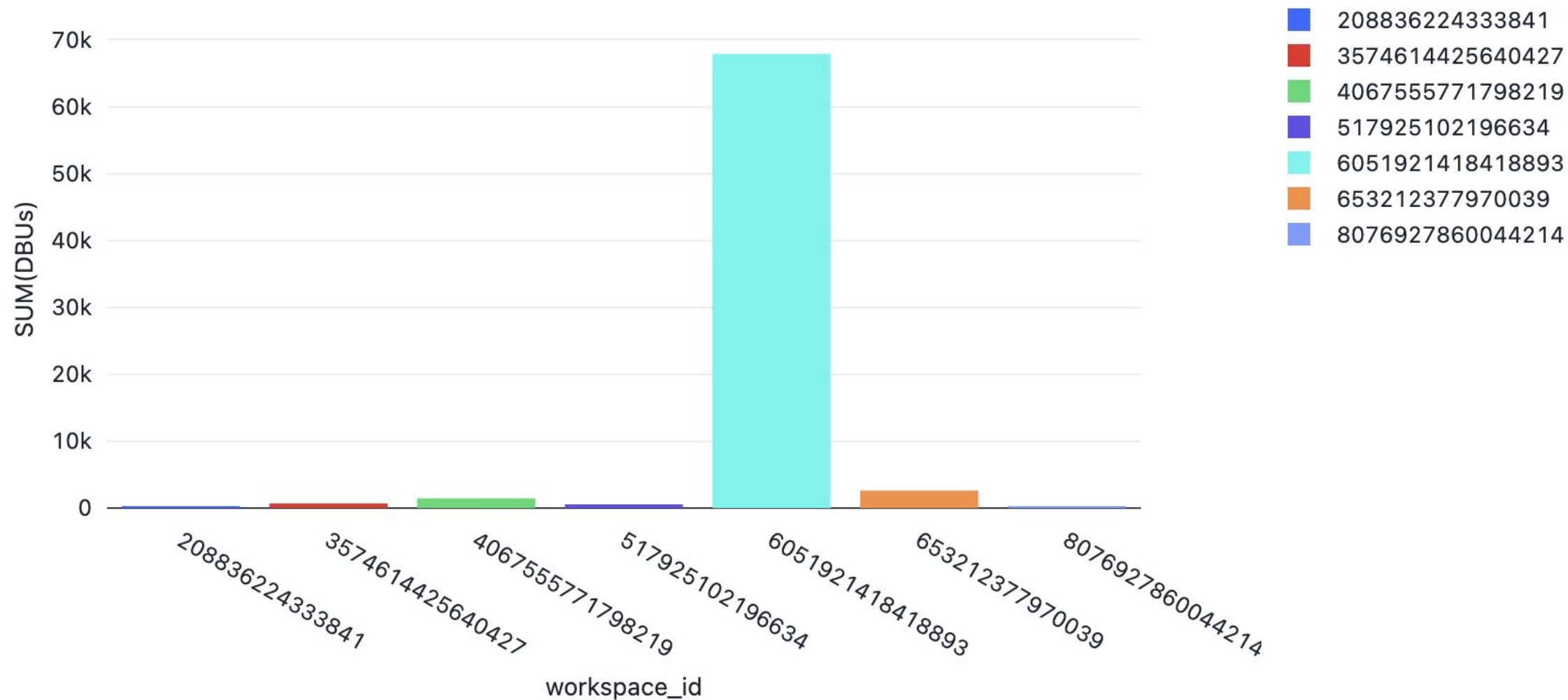
▸  _sqlidf: pyspark.sql.dataframe.DataFrame = [owner: string, workspace_id: string ... 2 more fields]

Table

Visualization 1 ▾

+

Preview



Clusters Core Team Data Science

[Configuration](#)
[Notebooks \(3\)](#)
[Libraries](#)
[Events](#)

Web Terminal

Web terminal provides a Bash terminal running in the

Web Terminal

RStudio Server

RStudio is a registered trademark of Posit Software, P

To use RStudio Server, you need to install the RStudio

Set up RStudio

Permission Settings for: Clusters Core Team Data Science Cluster

NAME	PERMISSION	
quinn.leng@databricks.com (quinn.leng@databricks.com)	Can Manage	✕
All Users	Can Manage	✕
Tarek Madkour (tarek.madkour@databricks.com)	Can Manage	✕
Admins	Can Manage inherited	

Select user, group or service principal...

Can Manage

+ Add

Cancel

Save

Dashboard



☆ Cost Management (powered by UC System Tables) - go/foresight/staging/costforecast

Share

Subscribe ▾

Refresh



Spend at a glance

\$DBUs spent (month to date)

\$239K

🔄 3 days ago

\$DBUs used (last full month)

\$215K

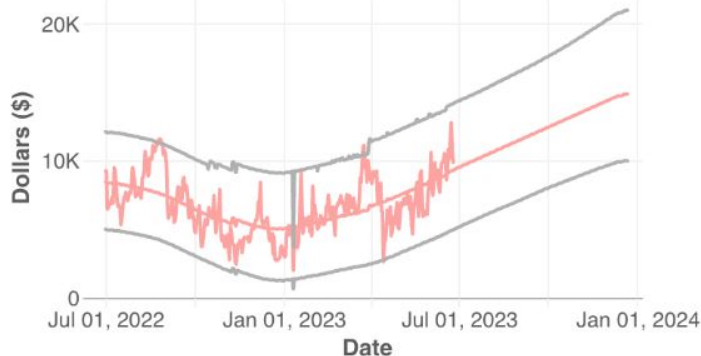
🔄 3 days ago

\$DBUs expected (this month)

\$200K

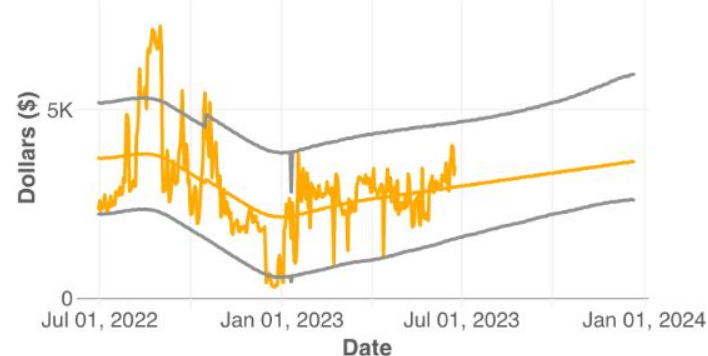
\$DBUs forecasts and Data Assets Utility

Forecast Total \$DBUs



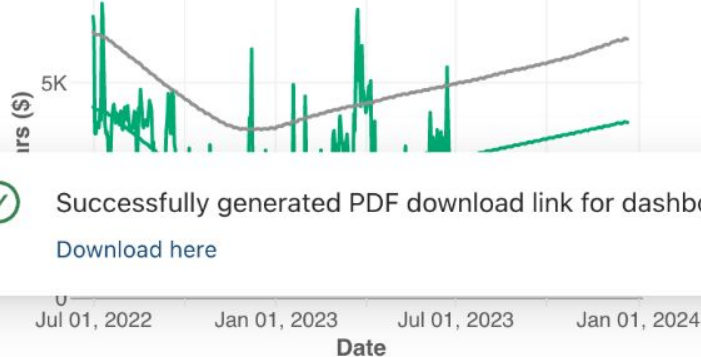
🔄 3 days ago

Forecast SQL \$DBUs



🔄 3 days ago

Forecast All Purpose \$DBUs

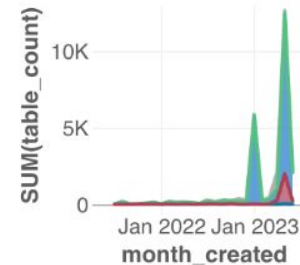


Forecast Model Inference \$DBUs



Tables/Volumes

Tables Created (MoM)



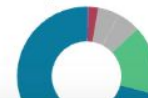
table_type:

EXTERNAL

🔄 3 days ago

Tables Created by Type

COUNT(table_catalog)



🔄 3 days ago



Successfully generated PDF download link for dashboard: Cost Management (powered by UC System Tables) - go/foresight/staging/costforecast

[Download here](#)

Budgets

Schema browser

hive_metastore > default

Filter tables & columns...

- 00_test_view
- 012345
- 0123456_a
- 067ec3da343e4592b3c7ea2...
- 0990607603c147b7a8a0466...
- 100m_lengthy_string
- 10g_1000_files_repartitions
- 10g_lengthy_string
- 10gb_lengthy_string
- 10gb_unpartitioned
- 10gb_unpartitioned_target
- 10k_lengthy_string
- 10m_lengthy_string
- 120mb_40_partitions
- 120mb_40_partitions_with_3
- 120mb_40_partitions_with_3...
- 120mb_40files_target
- 120mb_repartition
- 120mb_repartitions
- 15q2s
- 1a4533cce7374492b723a93...
- 1g_1000_files
- 1g_1000_files_repartition
- 1g_1000_files_repartitions
- 1g_lengthy_string
- 1g_partioned_files

Billing DBUs

Cancel hive_metastore.default

Data Team En... Serverless M Save Schedule Share

```
1 SELECT sum(dbus) / 400000 as dbu_util
2 FROM system.billing.billing_logs
3 WHERE date >= date_trunc('month', now())
```

Results

Executing

Query saved

Refreshed 3 minutes ago



Billing DBUs: dbu_util ... Share

Start by selecting the query that you would like to monitor using the search bar. Setup Instructions ?

Query

Billing DBUs

	Value column	Aggregate by	Condition	Threshold
Trigger when	dbu_util	None	>	0.90

Top row value is 0.66406235429... Only the first row of results is evaluated, so consider adding an aggregation or sorting your query.

When triggered, send notification

Just once

Template

Use default template

Refresh

Every hour

Advanced

SQL warehouse

✓ Data Team Endpoint

Serverless

M

Create alert

Billing DBUs: dbu_util > 0.90

STATUS: UNKNOWN

Alert condition has not been evaluated

Query [Billing DBUs](#)

Value column

Trigger when dbu_util

Top row value is 0.66406235429. This is not an aggregation or sorting your query

Notifications Notifications are sent just once, Set to default notification template

Refresh At 27 seconds past the minute, 27 seconds

SQL warehouse [Data Team Endpoint](#)

Add existing alert destinations

Search destinations and users...

bugbashr

asd#\$123asdasdsa

pd ML Data PagerDuty Low

product info dl

bugbash webhook

00TestMatterMost11

teams dummy

Create new destinations in [Alert destinations](#)

Cancel

Save

Alert destinations

alex.lichen@databricks.com

michael.milirud@databricks.com

+ Add

Billing DBUs

(TRIGGERED) Billing DBUs: dbu_util > .5

mail.google.com/mail/u/0/#inbox/FMfcgzGrbHqzhWmjWwFbkbSVGvDMZGSK

Search in mail

1 of 3,795

99+

Mail

Chat

Spaces

Meet

Compose

Inbox3,339

Starred

Snoozed

Sent

Drafts13

More

Labels

benefits

confluence13

design-docs79

eng-redash1,818

google docs com...381

market-info213

nephos

pm-info (custom...436

prds130

prod-staff (updat...609

Sensitive46

More

(TRIGGERED) Billing DBUs: dbu_util > .5

Databricks<noreply@databricks.com>to me

databricks

Your alert "Billing DBUs: dbu_util > .5" has been triggered

Go to alert

Go to query

Databricks: 160 Spear Street, 13th Floor, San Francisco, CA 94105 USA

© Databricks 2022. All rights reserved.

Reply

Forward

12:21 PM (21 minutes ago)

Create alert

 My alerts

Migrate



Billing DBUs: dbu_util > .5

Share Edit Refresh

STATUS: TRIGGERED

Last triggered 21 minutes ago

Destinations ?

+ Add

alex.lichen@databricks.com

Query [Billing DBUs](#)

	Value column	Condition	Threshold
Trigger when	dbu_util	>	.5

Top row value is 0.66406235429... Only the first row of results is evaluated, so consider adding an aggregation or sorting your query.

Notifications Notifications are sent just once, until back to normal.
Set to default notification template.

Refresh At 57 seconds past the minute, at 0 minutes past the hour

SQL warehouse [Data Team Endpoint](#)